遗传算法拟合三次多项式实验报告

摘要：基于 UCI Wine Quality（红葡萄酒）数据，使用遗传算法搜索三次多项式系数，并生成多种可视化和报告文档展示从数据预处理到结果分析的完整流程。

关键词：Wine Quality；遗传算法；三次多项式；均方误差；Python

1 案例与数据集

1.1 案例背景

本实验选用 UCI Wine Quality 红葡萄酒数据集作为研究对象。该数据集公开透明，包含 1599 条样本和 11 个理化特征以及 1 个质量评分，适合作为教学与实验案例。

1.2 数据准备

为了聚焦单变量多项式拟合，我们通过 Python 脚本从原始文件中抽取 alcohol 与 quality 两列，重命名为 x 与 y 并保存为 wine_alcohol_quality.csv。这样获得的 1599 组 (x, y) 样本覆盖 8%~14% 的酒精含量区间，评分集中在 5~7 分，为后续拟合提供了充足的噪声数据。

2 开源资源

实验实现完全基于开源软件栈：Python 3.11 负责脚本执行与文件操作；NumPy 用于数组计算、Horner 多项式以及 polyfit 基准回归；Matplotlib 既承担绘制散点、训练曲线、残差图的任务，也通过无界面后端生成 PDF 和图片。所有依赖均可在普通教学环境中安装。

3 实现方案

3.1 遗传算法设计

个体编码为四维向量 [a, b, c, d]，对应三次多项式 $y = ax^3 + bx^2 + cx + d$。实验中设置种群规模为 80，最大迭代代数为 300，并将系数范围限制在 [-5, 5] 以避免数值发散。适应度函数选用均方误差（MSE），通过 Horner 形式计算多项式以提升数值稳定性。

选择算子采用 k=3 的锦标赛选择，交叉算子为线性混合（blend crossover），突变算子在每个基因上叠加高斯噪声并在预设范围内裁剪。通过这一组合，算法能够在保证多样性的前提下逐步逼近最优解。

3.2 实验流程

完整流程包括：1) 读取数据并固定随机种子；2) 初始化种群并计算初始适应度；3) 在每一代中执行选择、交叉、突变并更新种群；4) 记录历代最佳 MSE，获得训练收敛曲线；5) 使用 numpy.polyfit 对同一数据做三次多项式最小二乘拟合作为对照；6) 保存系数、MSE、R^2 等指标，并输出多个图像文件。

4 结果分析

4.1 指标对比

遗传算法最终得到的多项式系数约为 [0.0059, -0.1239, 0.9592, 2.3111]，在测试数据上的 MSE 为 0.5227，对应的 R^2 约为 0.20。相比之下，numpy.polyfit 得到的三次多项式 MSE 为 0.4991，略优于 GA，但差距不大，说明 GA 在没有梯度信息的情况下仍能搜索到接近最优的解。

4.2 可视化与讨论

拟合曲线图表明，GA 拟合出的红色多项式曲线穿过蓝色散点的主要密集区域，反映出酒精含量与评分总体呈正相关关系，但在高酒精区间趋于饱和。训练收敛曲线显示误差在前几十代急剧下降，随后在 0.52 左右震荡收敛，验证了遗传算法的有效性。残差散点图显示误差整体围绕零线随机分布，仅在 13% 以上酒精度略有扩散，这提示若引入更多理化特征有望进一步提升拟合质量。

5 图表展示

本节集中展示三张关键图像，用于支持分析与答辩说明。

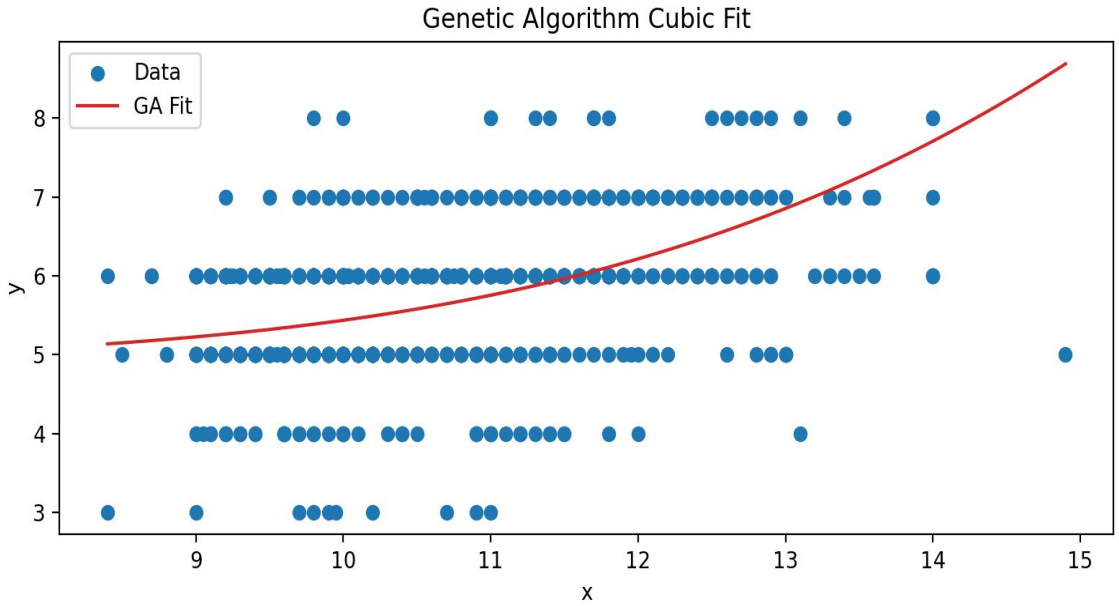

图 1 遗传算法三次多项式拟合效果

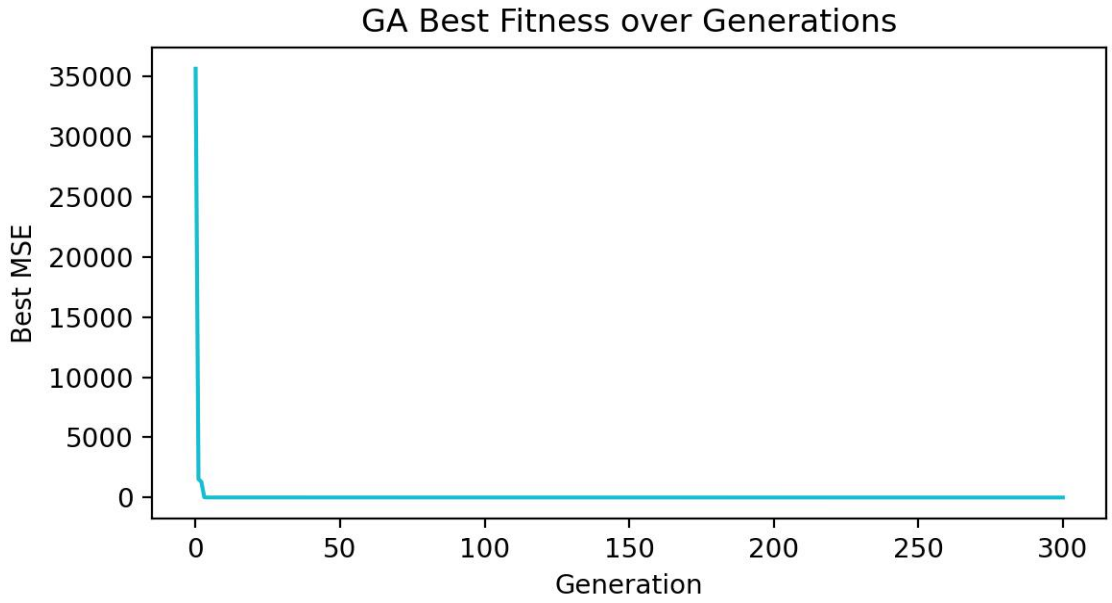

图 2 GA 训练收敛曲线

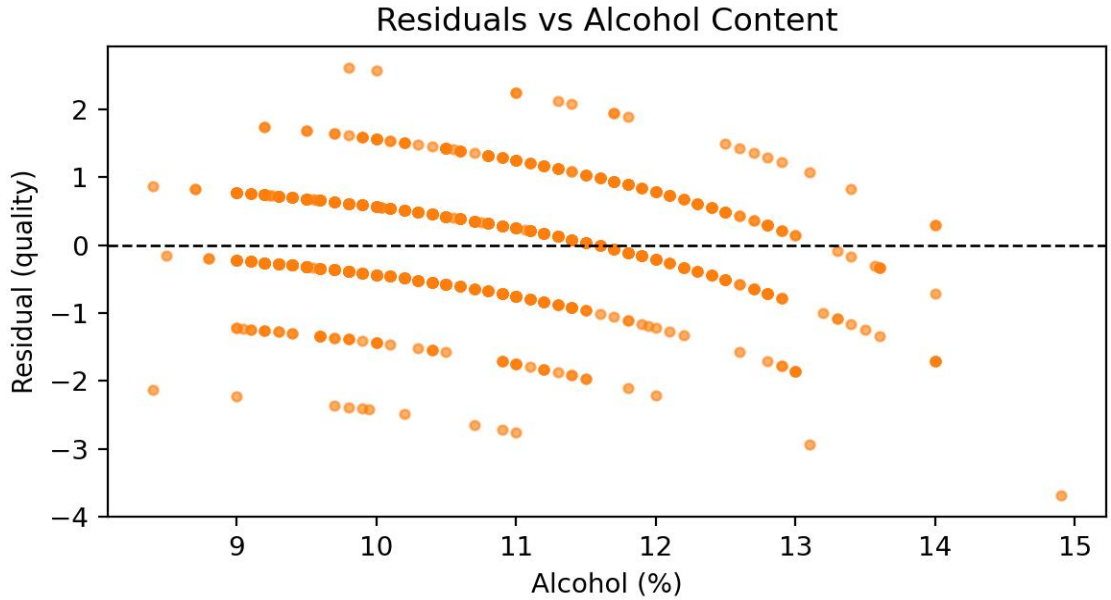

图 3 残差散点图

6 附录：带注释的源代码

```
"""Genetic algorithm to fit a cubic polynomial to noisy observations."""
from __future__ import annotations

import csv
import json
import math
import random
from dataclasses import dataclass
from pathlib import Path
from typing import List, Tuple

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np

DATA_PATH = Path(__file__).resolve().parents[1] / "data" / "wine_alcohol_quality.csv"
PLOT_PATH = Path(__file__).resolve().parents[1] / "results" / "ga_fit_plot.png"
METRIC_PATH = Path(__file__).resolve().parents[1] / "results" / "ga_metrics.json"


@dataclass
class GAConfig:
population_size: int = 80
generations: int = 300
crossover_rate: float = 0.8
mutation_rate: float = 0.2
mutation_scale: float = 0.5
coefficient_bounds: Tuple[float, float] = (-5.0, 5.0)
```

```
def load_dataset(path: Path) -> Tuple[np.ndarray, np.ndarray]:
xs, ys = [], []
with path.open() as f:
reader = csv.DictReader(f)
for row in reader:
xs.append(float(row["x"]))
ys.append(float(row["y"]))
return np.array(xs), np.array(ys)


def random_individual(bounds: Tuple[float, float]) -> np.ndarray:
low, high = bounds
return np.array([random.uniform(low, high) for _ in range(4)])


def evaluate_mse(individual: np.ndarray, xs: np.ndarray, ys: np.ndarray) -> float:
a, b, c, d = individual
preds = ((a * xs + b) * xs + c) * xs + d    # Horner form for stability
return float(np.mean((preds - ys) ** 2))


def tournament_selection(population: List[np.ndarray], fitness: List[float], k: int = 3) -> np.ndarray:
contenders = random.sample(list(zip(population, fitness)), k)
winner = min(contenders, key=lambda item: item[1])[0]
return winner.copy()


def blend_crossover(parent1: np.ndarray, parent2: np.ndarray, alpha: float = 0.5) ->
Tuple[np.ndarray, np.ndarray]:
child1 = alpha * parent1 + (1 - alpha) * parent2
child2 = alpha * parent2 + (1 - alpha) * parent1
return child1, child2


def mutate(individual: np.ndarray, cfg: GAConfig) -> np.ndarray:
for i in range(len(individual)):
if random.random() < cfg.mutation_rate:
individual[i] += random.gauss(0, cfg.mutation_scale)
individual[i] = max(cfg.coefficient_bounds[0], min(cfg.coefficient_bounds[1], individual[i]))
return individual


def run_ga(xs: np.ndarray, ys: np.ndarray, cfg: GAConfig) -> Tuple[np.ndarray, float, List[float]]:
population = [random_individual(cfg.coefficient_bounds) for _ in range(cfg.population_size)]
fitness = [evaluate_mse(ind, xs, ys) for ind in population]

best_ind, best_fit = min(zip(population, fitness), key=lambda item: item[1])
best_ind = best_ind.copy()
history: List[float] = [best_fit]
```

```
for gen in range(cfg.generations):
new_population: List[np.ndarray] = []
while len(new_population) < cfg.population_size:
parent1 = tournament_selection(population, fitness)
parent2 = tournament_selection(population, fitness)
if random.random() < cfg.crossover_rate:
child1, child2 = blend_crossover(parent1, parent2)
else:
child1, child2 = parent1.copy(), parent2.copy()
new_population.extend([mutate(child1, cfg), mutate(child2, cfg)])
population = new_population[:cfg.population_size]
fitness = [evaluate_mse(ind, xs, ys) for ind in population]
current_best_ind, current_best_fit = min(zip(population, fitness), key=lambda item: item[1])
if current_best_fit < best_fit:
best_ind, best_fit = current_best_ind.copy(), current_best_fit
history.append(best_fit)
if gen % 50 == 0:
print(f"Generation {gen}: MSE={best_fit:.4f}")
return best_ind, best_fit, history


def plot_results(xs: np.ndarray, ys: np.ndarray, coeffs: np.ndarray) -> None:
xs_dense = np.linspace(xs.min(), xs.max(), 200)
preds = np.polyval(coeffs, xs_dense)
plt.figure(figsize=(8, 4))
plt.scatter(xs, ys, label="Data", color="#1f77b4")
plt.plot(xs_dense, preds, label="GA Fit", color="#d62728")
plt.title("Genetic Algorithm Cubic Fit")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.tight_layout()
plt.savefig(PLOT_PATH, dpi=200)
plt.close()


def plot_training_history(history: List[float]) -> None:
plt.figure(figsize=(6, 3.5))
plt.plot(history, color="#17becf")
plt.title("GA Best Fitness over Generations")
plt.xlabel("Generation")
plt.ylabel("Best MSE")
plt.tight_layout()
out_path = PLOT_PATH.with_name("ga_training_curve.png")
plt.savefig(out_path, dpi=200)
plt.close()


def plot_residuals(xs: np.ndarray, ys: np.ndarray, coeffs: np.ndarray) -> None:
preds = np.polyval(coeffs, xs)
```

```
residuals = ys - preds
plt.figure(figsize=(6, 3.5))
plt.scatter(xs, residuals, s=12, alpha=0.6, color="#ff7f0e")
plt.axhline(0, color="black", linewidth=1, linestyle="--")
plt.title("Residuals vs Alcohol Content")
plt.xlabel("Alcohol (%)")
plt.ylabel("Residual (quality)")
plt.tight_layout()
out_path = PLOT_PATH.with_name("ga_residuals.png")
plt.savefig(out_path, dpi=200)
plt.close()


def compute_r_squared(preds: np.ndarray, ys: np.ndarray) -> float:
ss_res = float(np.sum((ys - preds) ** 2))
ss_tot = float(np.sum((ys - ys.mean()) ** 2))
return 1 - ss_res / ss_tot


def main() -> None:
cfg = GAConfig()
xs, ys = load_dataset(DATA_PATH)

best_coeffs, best_mse, history = run_ga(xs, ys, cfg)
baseline_coeffs = np.polyfit(xs, ys, 3)
baseline_mse = evaluate_mse(baseline_coeffs, xs, ys)
preds = np.polyval(best_coeffs, xs)
r_squared = compute_r_squared(preds, ys)

plot_results(xs, ys, best_coeffs)
plot_training_history(history)
plot_residuals(xs, ys, best_coeffs)

METRIC_PATH.write_text(json.dumps(
{
"ga_coefficients": best_coeffs.tolist(),
"ga_mse": best_mse,
"ga_r2": r_squared,
"ga_history": history,
"polyfit_coefficients": baseline_coeffs.tolist(),
"polyfit_mse": baseline_mse,
},
indent=2,
))
print("Best coefficients (GA):", best_coeffs)
print("Best MSE (GA):", best_mse)
print("Baseline coefficients (polyfit):", baseline_coeffs)
print("Baseline MSE (polyfit):", baseline_mse)


if __name__ == "__main__":
```

```
random.seed(1234)
np.random.seed(1234)
main()
```